Fichier réponse

TP1-Jour 1:

TP1-1:

Combien de temps à duré le fun ? Où voit-on les logs de chaque étapes?

Le run à duré 20 sec. Pour trouvé les logs il faut cliqué sur le push présent dans Actions puis cliqué sur le run pour avoir tout l'affichage.

TP1-4

Quel test à échoué ? À quelle ligne ? Qu'est ce qui a permis de le trouver rapidement ?

Le test test_is_valid_title_accepts_normal_title() à la ligne 13. Grâce à l'affichage des logs on peut déterminer simplement d'où vient l'erreur.

TP1-7

Pourquoi attend-on que la CI soit verte avant de fusionner ?

On attend afin que tout les modifications soient prise en compte

TP2-Jour 2:

TP2-1:

Quelle différence entre un Linter (flake8) et un formateur (black) ? Pourquoi séparer Lint et test en deux jobs ?

Flake8 détecte les erreurs dans le code (import non utilisé, variable non définie) alors que black défini un style dans la manière d'écrire les fichiers et détecte donc si tout les fichiers suivent cette manière sinon ils les reformatent.

Séparer Lint et test en deux jobs permet de séparer le métier de chacun, cela permet donc d'identifier quelle jobs provoque une erreur.

TP2-4:

Pourquoi tester sur plusieurs versions de python ? Que gagne-t-on avec le cache ? Un artefact, c'est quoi ?

On test sur plusieurs versions de python pour savoir si le code ne casse pas sur les différentes versions, on veut être sur qu'il est utilisable sur les versions différentes sur lesquelles il peut être utilisé.
Le cache permet d'aller plus vite et pour cela on n'a pas besoin de retélécharger les dépendances.
Un artefact est un fichier produit par un job et sauvegardé à la fin du job, pour pouvoir le télécharger ou le réutiliser dans une étape suivante

TP2-5:

Pourquoi interdire de pousser directement sur la main ?

On interdit de pousser directement sur la main pour éviter de push un code qui ne passe pas les tests donc qui risque de tout casser.

TP2-8:

Pourquoi git ne peut-il pas résoudre ce conflit seul ? Qu'est ce qui a vérifié le code après la résolution ?

Git ne peux pas résoudre le conflit seul car afin que tout soit résolu il faut soit suprimmer du code soit modifier, donc git ne sait pas ce qu'il faut changer.
Après la résolution le code est add commit puis push. Dans Github la rule que j'ai mis en place vérifie que tout est passé.

TP2-9:

Pourquoi GitHub masque-t-il le secret ? Que se passe-t-il si on le commit dans le dépôt ? Secret ou variable: comment choisir ?

GitHub masque le secret pour ne pas rendre publique des token par exemple. Tout les commit publié sur GitHub sont visibles par tous et pour toujours. Si on le commit dans le dépôt il est masqué avec des ****. Les variables sont des données peu sensibles que l'on va souvent utiliser et que l'on a pas besoin de cacher alors que le secret sont des données que l'on va utiliser plus souvent mais qui sont beaucoup plus sensible comme des token c'est pour cela qu'il faut les cacher.

TP2-11

pipeline_A: mauvaise indentation
pipeline_B: manque de uses pour avoir accès au fichier des dossiers
pipeline_C: manque des quote pour les python-version

Pour remedier à tout ça il faut bien relire et utiliser des rules.